

Lab 1: Logic Gates

Prepared by

Student name: Aryan Qureshi

Student ID: 300467938

Student name: Borna Chadegani

Student ID: 300488971

Group # 10

ITI 1100 Digital Systems I

Professor: Dr. Aneta Traikova, PhD, P. Eng.

Presented to:

TA and Lab Supervisor:

Dorsa Khodabakhsh (dkhod005@uottawa.ca)

Rahil Ghalamboland (rghal080@uottawa.ca)

Lauren Hendley (lhend093@uottawa.ca)



uOttawa

School of Electrical Engineering and Computer Science

École de science informatique et de génie électrique

Faculty of Engineering | Faculté de génie

Summary

This experiment was conducted to figure out the behavior of basic and combined logic gates (or digital circuits) used in digital systems. Simple and combinational logic circuits were constructed using Quartus II software including built in logic nodes, then tested to observe input-output relationships for a given circuit. Truth tables were then generated to determine the logical function of each circuit and to identify equivalent logic gate configurations. Overall it shows how complex and more larger circuits can be created by interconnecting basic ones.

Introduction

Background

Digital systems form an important foundation of modern computers and electronic devices. Logic gates and circuits are at the core of these systems which are basic electronic building blocks that perform boolean operations on binary inputs. Logic gates in fact operate using two discrete voltage levels that represent logical values of 0 (indicating false) and 1 (indicating true). By combining logic gates in different configurations and placements, complex circuits capable of performing arithmetic, data processing and control operations can be constructed.

Pre Lab

The objective of this pre lab was to analyze basic combinational logic circuits by deriving their Boolean logic expressions, constructing truth tables, and identifying their equivalent logic gate functions. This was completed prior to the simulation in Quartus.

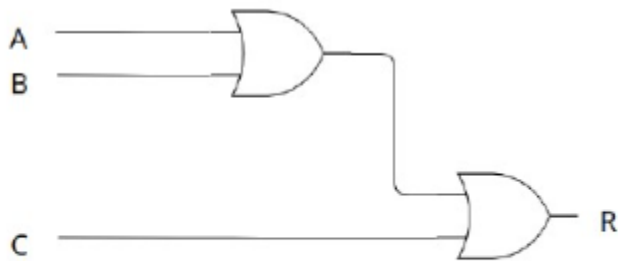


Figure 5.1.1: One-chip logic circuit

The circuit shown in Figure 5.1.1 consisted of two OR gates connected in series, with inputs A and B applied to the first OR gate and input C applied to the second OR gate.

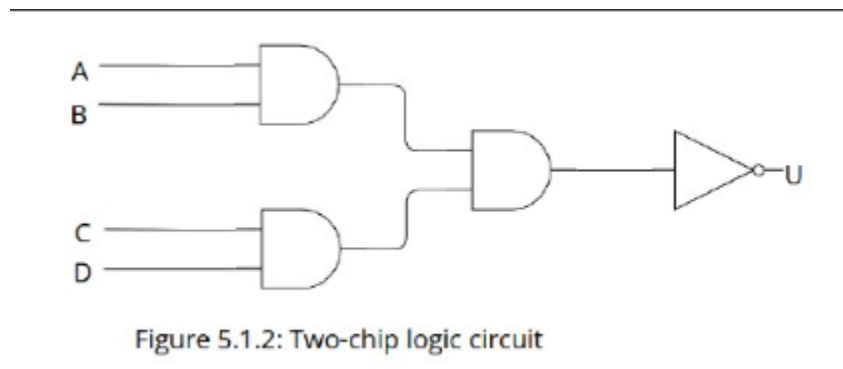
From the circuit, the Boolean logic expression was derived as: $R=A+B+C$

Truth Table
Figure 5.1.1

A	B	C	R
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

$R = A + B + C$

A truth table was constructed by evaluating all possible combinations of inputs A, B, and C. The results showed that the output R was logic high whenever at least one input was high and logic low only when all inputs were low. This behavior confirmed that the circuit was functionally equivalent to a three-input OR gate .



The circuit shown in Figure 5.1.2 consisted of two AND gates going into a third AND gate, followed by a NOT gate at the output. Inputs A and B were applied to the first AND gate, while inputs C and D were applied to the second AND gate.

The Boolean expression for the circuit was derived as:

$$\text{AND1} = A \cdot B$$

$$\text{AND2} = C \cdot D$$

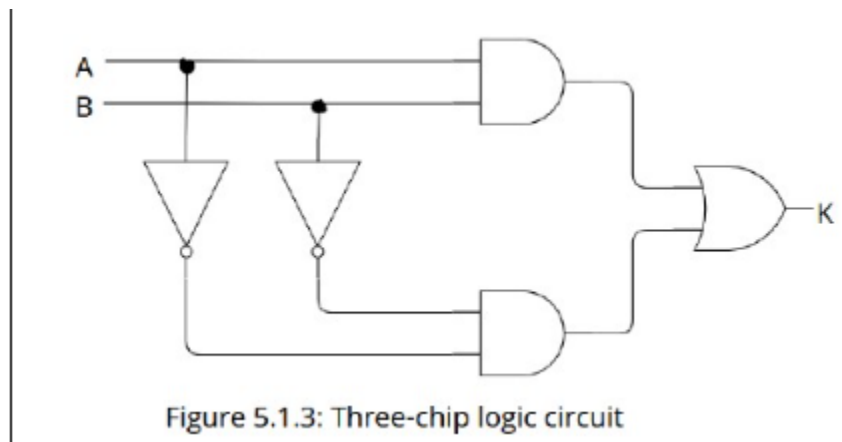
$$R = (A \cdot B) (C \cdot D)$$

Truth Table figure 5.1.2

A	B	C	D	R
0	0	0	0	1
0	0	0	1	1
0	0	1	0	1
0	1	0	0	1
1	0	0	0	1
0	0	1	1	1
0	1	0	1	1
1	0	0	1	1
0	1	1	1	1
1	0	1	1	0
1	1	1	1	0
1	0	1	0	1
1	1	0	1	1
1	1	1	0	1
1	0	1	0	1
1	1	0	0	1

$AND1 = A \cdot B$
 $AND2 = C \cdot D$
 $AND3 = AND1 \cdot AND2 = A \cdot B \cdot C \cdot D$
 $R = \text{not}(AND3) = \neg(A \cdot B \cdot C \cdot D)$
 $\therefore R = \neg(A \cdot B \cdot C \cdot D)$

A truth table was constructed for all combinations of A, B, C, and D. The output R was found to be logic low only when all four inputs were high, and logic high otherwise. This confirmed that the circuit was equivalent to a four-input NAND gate .



The circuit in Figure 5.1.3 used NOT gates to generate inverted inputs, followed by two AND gates and a final OR gate. Inputs A and B were each split into direct and inverted paths.

From the circuit, the Boolean expression was derived as:

$$K = (A \cdot B) + (A \cdot B)$$

A	B	K
0	0	1
0	1	0
1	0	0
1	1	1

• output First NOT Gate: $\neg A$
 • Second NOT gate output: $\neg B$
 • First AND Gate input: A (direct), B (direct)
 • output is $A \cdot B$
 • Second AND gate inputs: $\neg A$, $\neg B$
 output is: $\neg A \cdot \neg B$
 • final OR gate inputs: $A \cdot B$ (First AND)
 $\neg A \cdot \neg B$ (Second AND)
 • $K = (A \cdot B) + (\neg A \cdot \neg B)$
 $\hookrightarrow K = A \oplus B \rightarrow K = 1$ when A and B are both, 0 or 1.
 $\rightarrow K = 0$ when $(A = 0, B = 1)$ or $(A = 1, B = 0)$

A truth table was constructed for inputs A and B. The output K was high only when the inputs were different and low when the inputs were the same. This behavior confirmed that the circuit implemented an XOR function.

In Lab

In this lab, four specific circuit configurations (figures 5.1.5 through 5.1.8) were analyzed. These circuits were designed and simulated using Intel Quartus software to observe input-output relationships. Furthermore, the designs were implemented on the Altera DE2 - 115 Board for physical verification if it is needed. By generating truth tables and analyzing simulation waveforms, the functional behavior of each circuit was determined and compared to theoretical logic operations.

Objective of the report

The primary objective of this report is to experimentally analyze and study simple and combinational logic circuits. Specific goals include:

- Constructing combinational logic circuits from schematics using Quartus II Software.
- Determining the functional operation of these circuits through experimental testing and simulating.
- Identifying equivalent logic gate configurations by comparing resulting truth tables to known gate behaviors.
- Interconnecting various gates to create and analyze more complex logic functions.

Equipment

Hardware

- **Altera DE2-115 Development and Education Board**
Used to implement and test combinational logic circuits
Toggle switches used as logic inputs
LEDs used as logic outputs
 - **USB-Blaster Programming Cable**
 - Used to program the FPGA on the DE2-115 board

Software:

- **Intel Quartus Prime**
 - Used to design logic circuits using schematics
 - Used to compile, simulate, and program.

Logic Components:

- AND gates
- OR gates
- NOT gates
- NAND gates
- NOR gates
- XOR gates

Input and Output Devices:

- Toggle switches on the DE2-115 board for input variables (A, B, C, D)
- LEDs on the DE2-115 board for displaying output logic states

Additional Equipment

- Laboratory computer capable of running Quartus
- USB cable for board connection

Design Procedure

The purpose of this experiment was to examine and validate the combinational logic circuits by designing them using the schematic method, creating truth tables, and simulating the circuits.

For each of the circuits described in Figures 5.1.5 to 5.1.8, a new project was established in Intel Quartus Prime as per the naming convention. The schematic was employed to create each of the logic circuits using basic logic gates such as AND, OR, NOT, NAND, NOR, and XOR, depending on the requirements of each schematic.

Circuit Design

The logic gates were arranged and connected as per the requirements of each figure. Input pins were assigned for each of the logic variables (A, B, C, and D as required), and output pins were assigned to represent the output of each circuit.

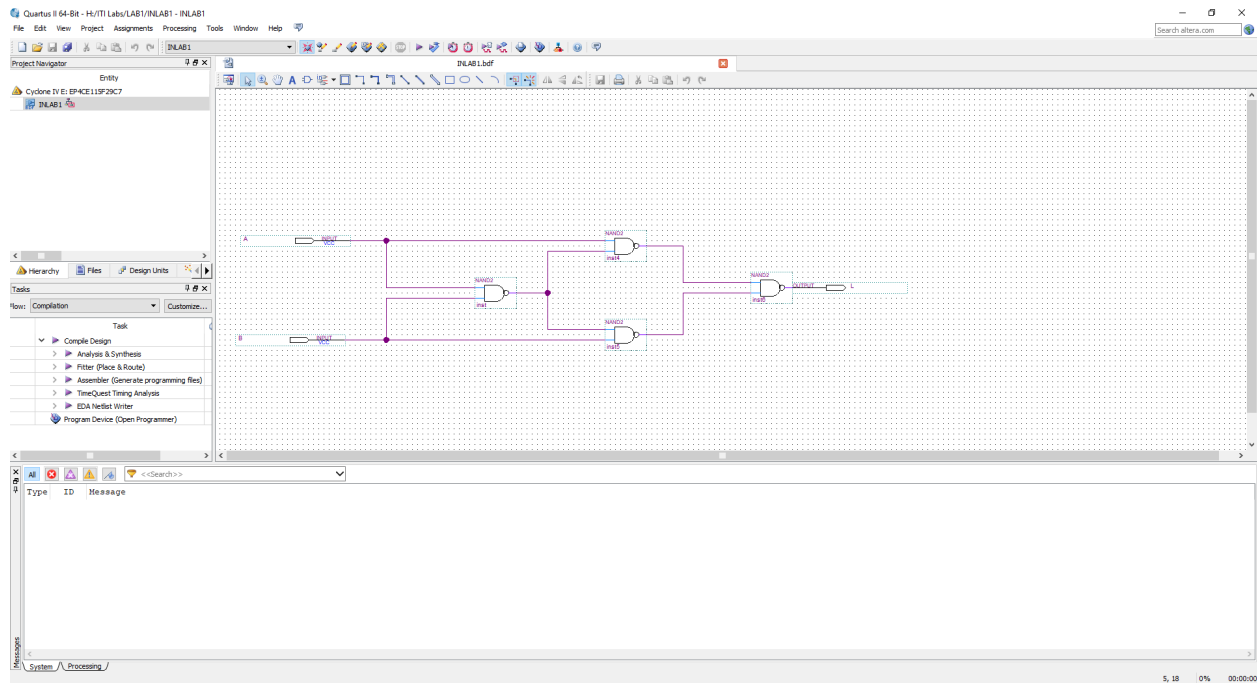


FIGURE 5.1.5

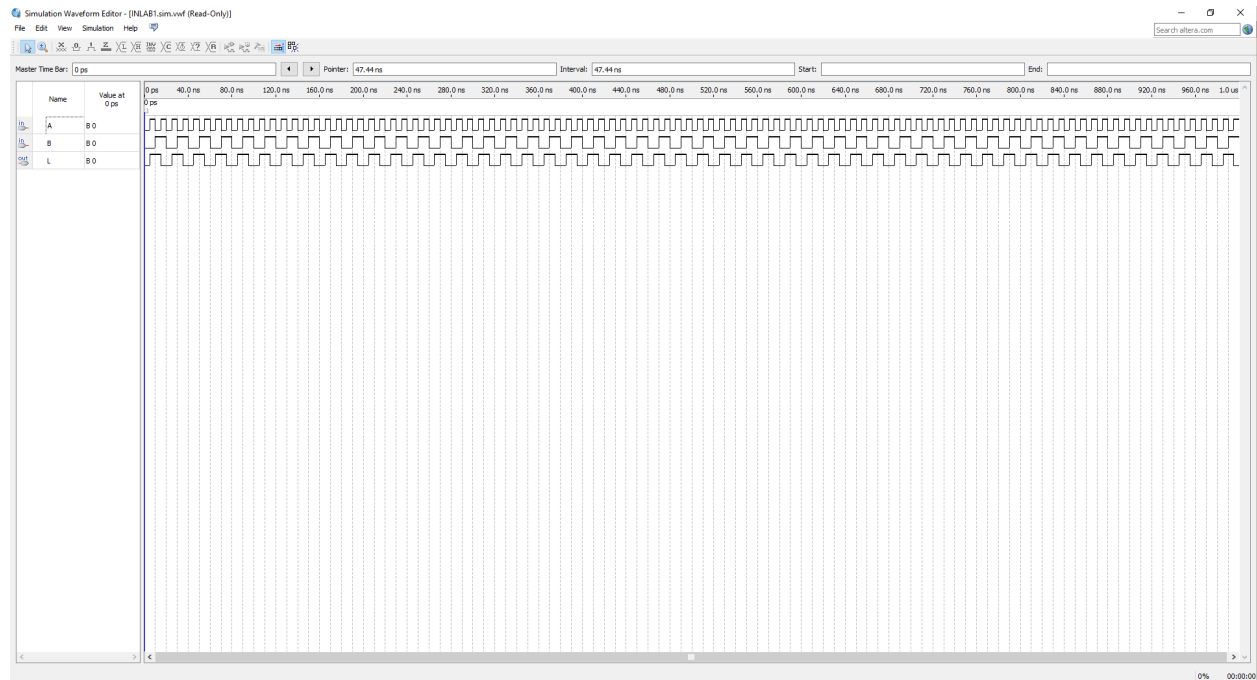


Figure 5.1.5- Waveform

The circuit shown in Figure 5.1.5 was implemented using NAND gates arranged to create an XOR function. Inputs A and B were applied as periodic square waves with different frequencies to generate all possible input combinations.

From the simulation waveform (Figure 5.1.5 – Waveform), it was observed that the output L was high only when exactly one of the inputs was high. Specifically:

When $A = 0$ and $B = 0$, the output remained low.

When $A = 1$ and $B = 0$, the output transitioned high.

When $A = 0$ and $B = 1$, the output transitioned high.

When $A = 1$ and $B = 1$, the output returned low.

These results confirmed correct XOR behavior, as the output changed state only when the inputs were different.

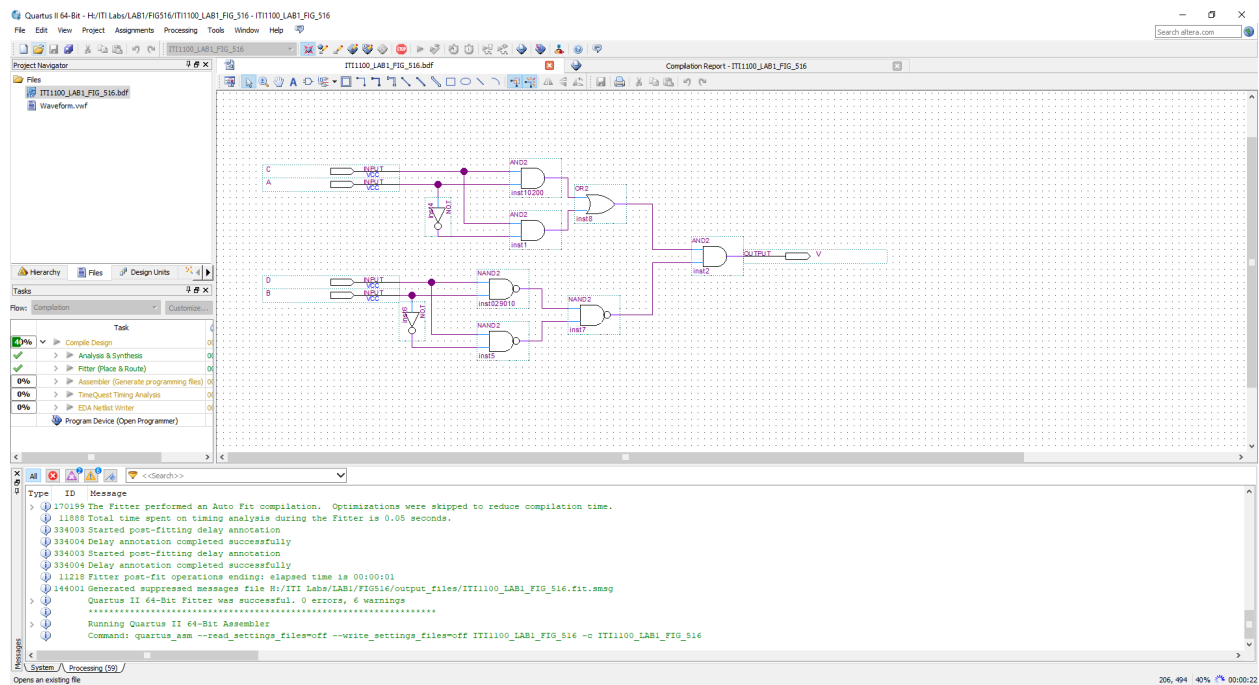


Figure 5.1.6

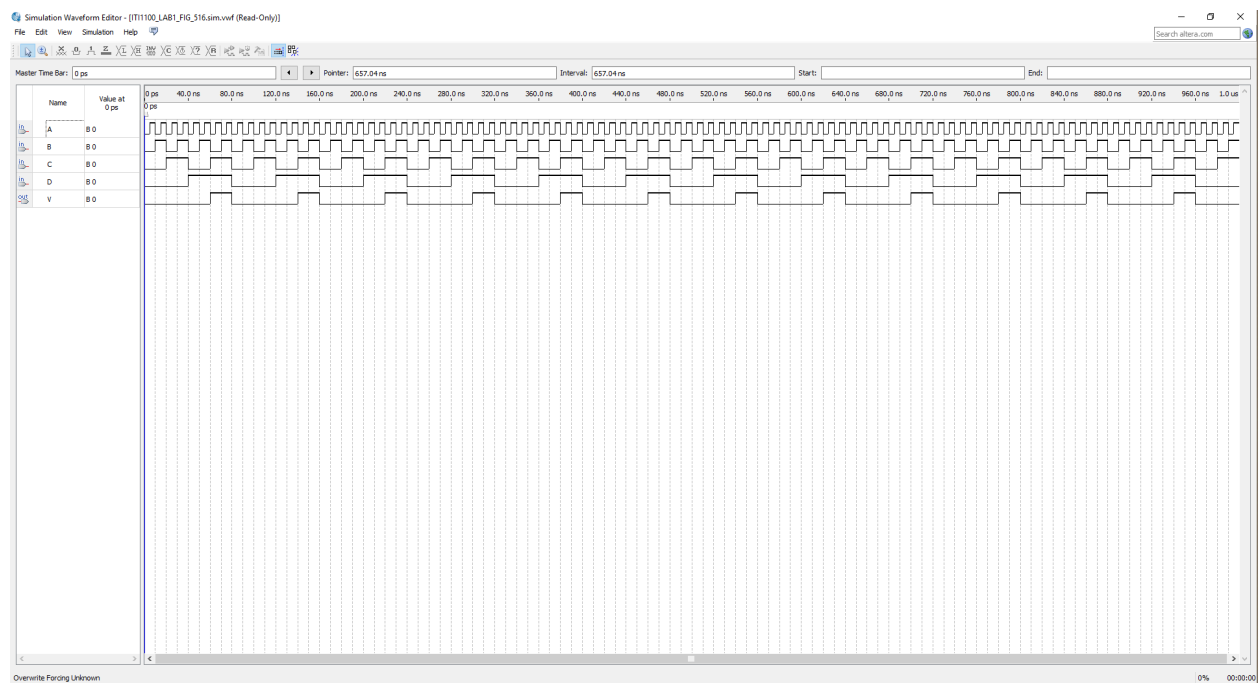


Figure 5.1.6 - Waveform

The circuit in Figure 5.1.6 consisted of multiple AND, NAND, NOT, and OR gates, with four inputs (A, B, C, and D). The logic was designed to produce an output only when specific combinations of the inputs were met.

From the simulation waveform (Figure 5.1.6 – Waveform), it was observed that the output V remained low for most input combinations. The output transitioned high only when the required logical conditions defined by the AND gate were met. Changes in each input that did not satisfy the full logic condition did not affect the output. This behavior demonstrated that the circuit correctly enforced multiple logical constraints before asserting the output, consistent with the expected AND expression.

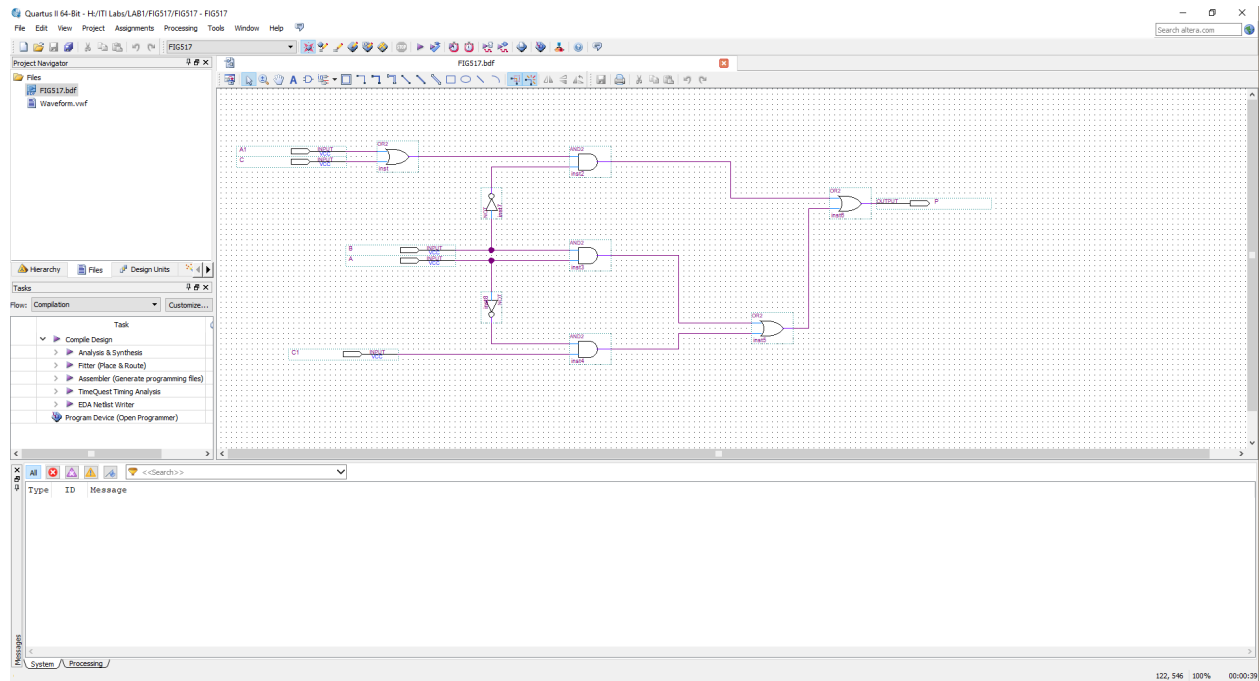


Figure 5.1.7

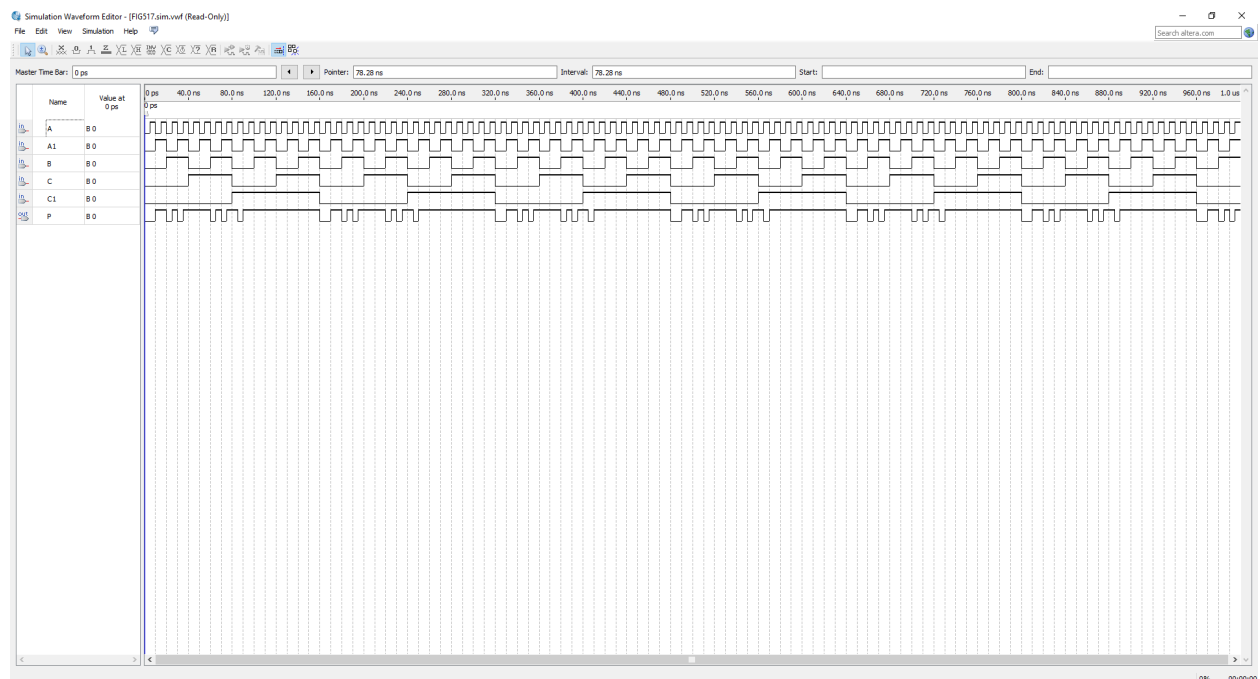


Figure 5.1.7 - Waveform

The circuit in Figure 5.1.7 was designed using a combination of OR, AND, and NOT gates. Inputs A, B, and C were applied as signals to observe the effect of each input on the output P.

From the simulation waveform (Figure 5.1.7 – Waveform), the observations were that the output P transitioned high whenever at least one valid logic path to the OR gate was active. The output remained low only when none of the required input conditions were satisfied. Inverted inputs correctly blocked certain logic paths, preventing unintended outputs. These observations confirmed that the circuit behaved as an OR logic function with selective gating, matching the expected operation from the truth table.

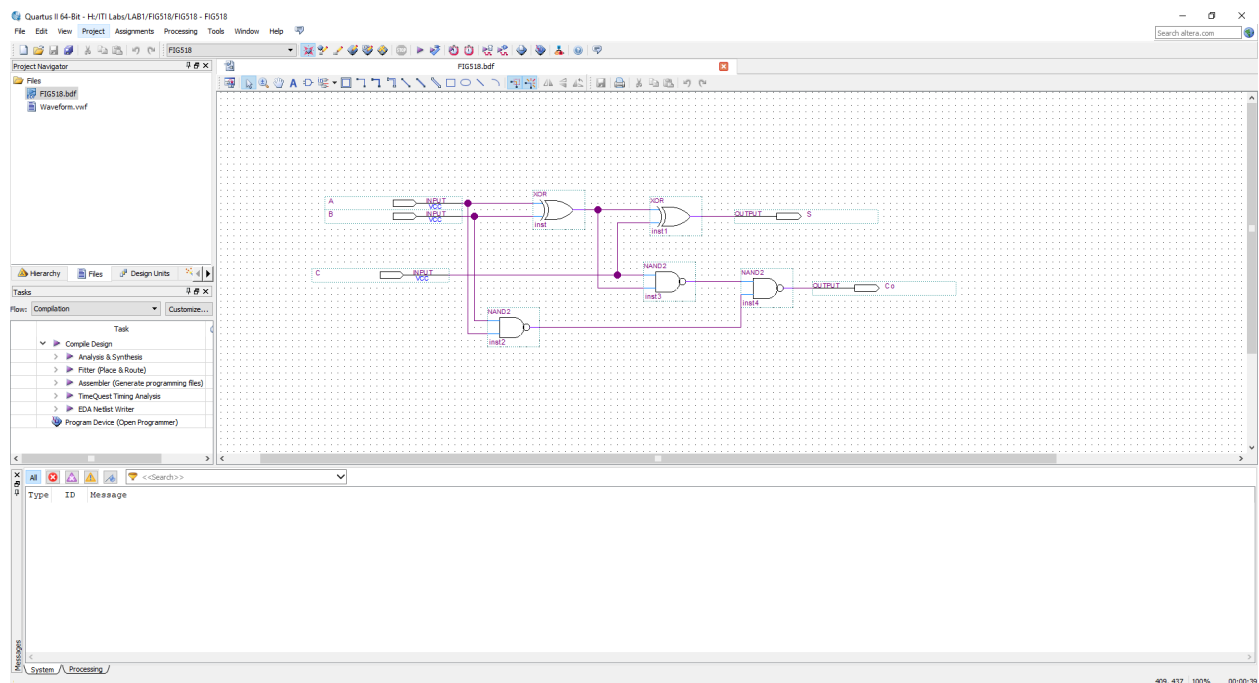


Figure 5.1.8

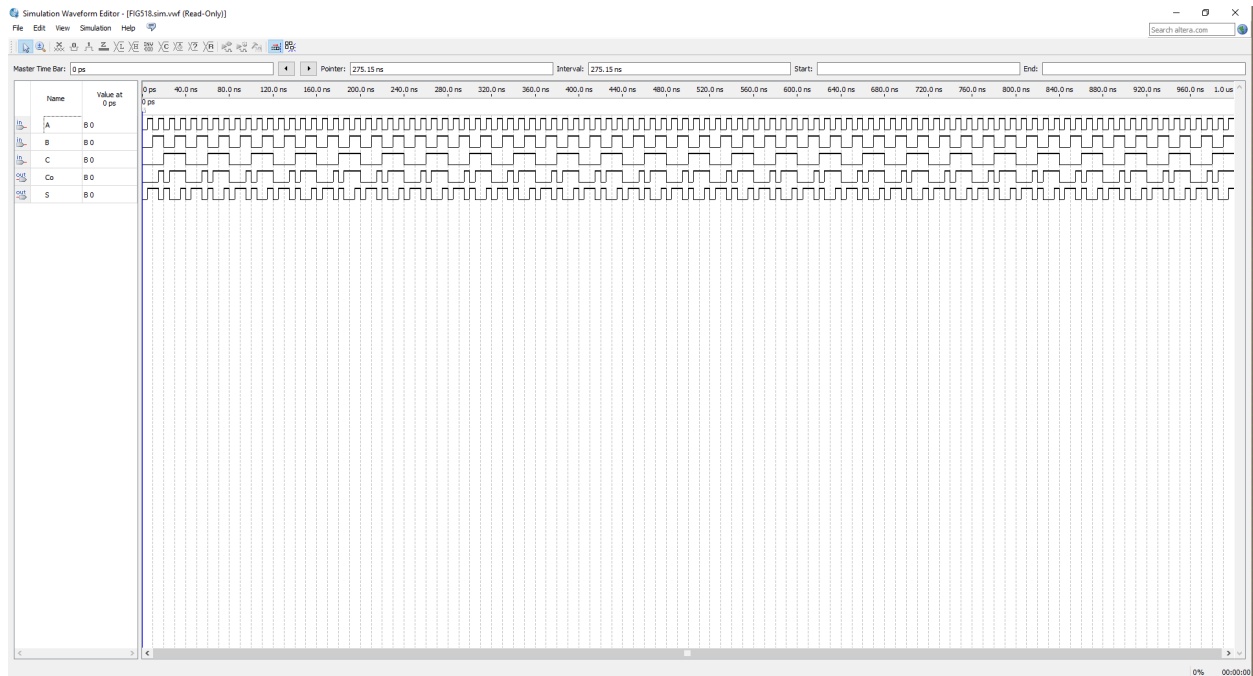


Figure 5.1.8- Waveform

The circuit in Figure 5.1.8 produced two outputs, S and Co, using XOR and NAND gates. Inputs A, B, and C were varied simultaneously during simulation.

From the simulation waveform (Figure 5.1.8 – Waveform), it was observed that output S toggled whenever an odd number of inputs were high, consistent with XOR gates. Output Co transitioned high when multiple inputs were simultaneously high. Both outputs responded to input changes, confirming combinational behavior. The simultaneous observation of both outputs demonstrated correct multi output logic operation.

Across all circuits, the simulated waveforms matched the expected behavior derived from manual analysis and truth tables. Each circuit responded correctly to changing inputs, and no unexpected output transitions were observed. This confirmed that the combinational logic designs were correctly implemented and functioned as intended.

Discussion of Results

The experimental results obtained through the simulations consistently confirmed the theoretical behavior of the combinational logic circuits analyzed in Figures 5.1.5 through 5.1.8. Each circuit successfully demonstrated how specific logic functions such as XOR, AND, and OR can be realized through different gate configurations.

The simulation waveform served as a primary confirmation tool to compare theoretical truth tables constructed before running the simulation, to the waveform to verify its correctness. For

instance the implementation of Figure 5.1.5 confirmed XOR behavior, as the output L was high only when A and B were at different logic levels. Similarly the multi output circuit in Figure 5.1.8 successfully demonstrated complex logic, where output S acted as an XOR for three inputs and Co responded when multiple inputs were high at the same time.

Figures 5.1.6 and 5.1.7 revealed that certain combinations of NOT and AND gates functioned identically to a NOR gate. This aligns with De Morgan's Law, proving that complex arrangements of these gates can still often be simplified to a single standard gate while maintaining the same logical output.

The identical nature of the waveform simulations and theoretical logic confirmed the value of Boolean Algebra in predicting circuit behavior. By analyzing the logic path from inputs to outputs, it was possible to precisely anticipate how the circuit would react to various input combinations before software testing.

Conclusion

This lab focused on the construction and analysis of simple combination logic circuits using basic logic gates. By experimentally testing different combinations and recording their outputs via the Intel Quartus software, the logical behavior of each circuit was first determined using boolean algebra starting from the inputs all the way to the outputs and then using truth tables to confirm whether or not the output matched that of each circuit's waveform graphs. The results of the waveforms appeared consistent with theoretical expectations, confirming the correct operation of standard logic gates and showing how combinations of these gates can implement specific boolean functions.

Through this experiment, equivalent logic gate configurations were identified, highlighting the concept that multiple circuit designs can produce the same logic output. Circuits like Figures 5.1.6 and 5.1.7 have NOT gates that the input passes through then leads to an AND gate which works as a NOR gate according to De Morgan's Law. This is a classic example in this lab where 2 combinations of logic gates can just be replaced with a NOR gate which serves the same purpose and also outputs the same result. Overall the results of finding these equivalent gates lead to the idea of how Boolean algebra principles can be applied to predict circuit behavior before experimental testing.

Ultimately, the experiment reinforced the value of Boolean analysis and truth tables as essential tools for designing and understanding digital systems. The ability to construct complex functions from basic building blocks and identify more efficient configurations that result in the same outputs, is a fundamental skill in digital logic design.

Appendices

Appendix A – Figure 5.1.5 (XOR Circuit)

- Figure 5.1.5-Schematic.PNG – Schematic diagram created using Quartus Block
- Figure5.1.5-Waveform.PNG – Simulation waveform verifying XOR behavior
- INLAB1.bdf – Quartus schematic file
- INLAB1.sim.vwf – Simulation waveform file
- INLAB1.msim.vwf – ModelSim waveform file
- INLAB1.msim.vcd – Value Change Dump file

Appendix B – Figure 5.1.6 (AND Circuit)

- ITI1100_LAB1_FIG_516.bdf – Quartus schematic file
- ITI1100_LAB1_FIG_516.qpf – Quartus project file
- ITI1100_LAB1_FIG_516.qsf – Quartus settings file
- ITI1100_LAB1_FIG_516.qws – Quartus workspace file
- Figure5.1.6-Schematic.PNG – Schematic diagram
- Figure5.1.6-Waveform.PNG – Simulation waveform
- Waveform.vwf – Quartus simulation waveform file

Appendix C – Figure 5.1.7 (OR Circuit)

- FIG517.bdf – Quartus schematic file
- FIG517.qpf – Quartus project file
- FIG517.qsf – Quartus settings file
- FIG517.qws – Quartus workspace file
- Figure5.1.7-Schematic.PNG – Schematic diagram
- Figure5.1.7-Waveform.PNG – Simulation waveform
- Waveform.vwf – Quartus simulation waveform file

Appendix D – Figure 5.1.8 (Multiple Output Circuit)

- FIG518.bdf – Quartus schematic file
- FIG518.qpf – Quartus project file
- FIG518.qsf – Quartus settings file
- FIG518.qws – Quartus workspace file
- Figure5.1.8-Schematic.PNG – Schematic diagram
- Figure5.1.8-Waveform.PNG – Simulation waveform
- Waveform.vwf – Quartus simulation waveform file

Appendix E – Pre Lab Material

- Logic and Truth Tables PreLab1.pdf – Pre-lab Boolean expressions and derived truth tables

Appendix F – Simulation and Verification Files

- Quartus Simulation Waveform Editor files (.vwf)
- Waveform and VCD files used for functional verification
- Screenshots of simulation results used for comparison with truth tables

Appendix G – Software Environment

- Intel Quartus
- Quartus Block Diagram/Schematic Editor
- Quartus Simulation Waveform Editor